

Documentación sobre Camus LDA:

Este documento se refiere a la documentación oficial de *Camus LDA*, carpeta contenedora con el proyecto Full-Stack.

Contexto:

Es un proyecto Full - Stack que propone un sistema de gestión de control interno de empresa de alcantarillado, que les permita saber quiénes son sus clientes, que trabajos tienen pendientes, cuanto material le queda en la bodega y, más adelante, facturar y sacar reportes al respecto.

Idea general en una frase:

El usuario entra a una página web → la página pide datos al servidor → el servidor los busca o guarda en PostgreSQL → la página los muestra.

Partes del proyecto:

- Vistas (camus LDA/src/): corresponde a la carpeta que contiene todo lo que se ve en el navegador.
 - **Tecnología:**
 - **React** → arma las pantallas con piezas reutilizables (componentes)
 - **Vite** → Herramienta que levanta el proyecto en desarrollo (Localhost:5173).
 - **Tailwind** → estilos (colores, tamaños, diseño).
 - **Zustand** → “memoria” de la app (listas de clientes, órdenes abiertas, operaciones de terreno, reportes) refiere al lugar donde la app guarda datos mientras la usas, no tiene que ver con la base de datos.
- Lógica (camus LDA/server): corresponde a la carpeta que contiene los controladores y servicios que reciben peticiones, aplica reglas y “habla” con la base de datos.
 - **Tecnologías:**
 - **Node.js + Express** → servidor que escucha en **localhost:4000**.
 - **Prisma** → traductor entre Typescript y PostgreSQL.
 - **PostgreSQL** → Base de datos.

Nota: Los puertos de las vistas y lógica son distintos porque son dos servidores independientes, uno entrega la interfaz y el otro la API. Permitiendo reiniciar, depurar y reiniciar cada parte por separado.

Flujo de acción:

Pantallas	Visual de lo disponible para el usuario.
Axios	Servicios de vistas.
Lógica	Recepciona peticiones del usuario.
Controladores	Toma las peticiones y las reparte a los servicios correspondientes.
Servicios de lógica	Servicios encargados de ejecutar peticiones.
Prisma + Postgre	Base de datos que almacena la información de gestión y su traductor.
Mock Data	Mímicas de datos y conexiones con servicios pagos.

Organización de vistas:

- **Src/Features:** Esta carpeta contiene módulos como:

- Auth.
- Dashboard.
- clientes.
- órdenes.
- inventario.
- facturación.
- reportes.
- usuarios.
- operaciones en terreno.
- Ayuda y configuración.

A su vez, en cada módulo suele haber:

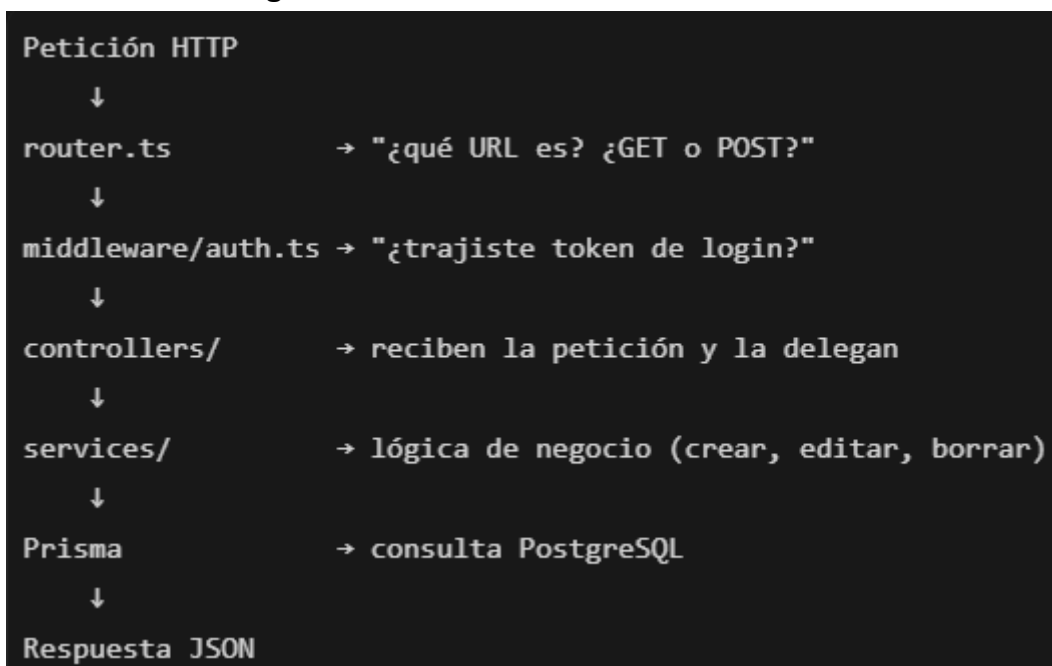
- **Components/** → Piezas como tablas, modales y formularios.
- ***Page.tsx** → pantalla principal.
- **Utils/** → Cálculos y exportaciones.

- **Src/components/:** Piezas reutilizables:

- **Layout/** → menú, header, footer (básicamente el “marco” de la app).
- **ui/** → botones, modales, tablas genéricas.

- **Src/store/**: guarda datos de la app y estado de pantalla; los datos reales vienen del backend vía API, no directamente de PostgreSQL.
- **Src/services/**: Puente a la lógica, archivos que llaman a las APIs. Por ejemplo
 - ClientServices → API Clientes
 - OrderServices → API órdenes de trabajo.
 - AuthServices → Hace conexión con el Login.
- **Src/Data/Mock/**: Corresponden a los datos de prueba.

Organización de la Lógica:



Carpetas Importantes:

- **Controllers/** → Puerta de entrada de las peticiones HTTPS (las delega a los servicios).
- **Services/** → Lógica real de los servicios del negocio.
- **Middleware** → Seguridad (JWT).
- **DB/** → Conexión a la base de datos.
- **External/** → Servicios externos que están simulados (Mirar diagrama de componentes).
- **Prisma/** → Esquema de tablas y datos iniciales (traductor de la base de datos).

Base de datos: esta carpeta, denominada como '*Prisma*', carga en postgresQL, los datos de demostración que se almacenan en *src/data/mock*. Si queremos ver los datos desde la propia BD, debemos abrir PostgreSQL.

Nota: La app PostgreSQL se abre al momento de iniciar windows, por lo que al querer correr la WebApp no es necesario iniciar el proceso de PostgreSQL.

Dentro de las tablas de datos encontramos las siguientes:

- Cliente.
- Usuario.
- Insumo.
- Orden de trabajo.
- Factura.
- Bitácora de auditoría.

Cosas no implementadas de verdad:

- Dashboard → Mocks.
- Biling → Pantalla + Mocks.
- Reports → Backend Parcial / Mocks de vistas.
- Usuarios → Pantalla + Mocks.
- Ops en terreno → Pantalla + Mocks.

¿Cómo se hizo?

A partir de la carpeta *legado_fastApi_sqlite* que utiliza:

- Python + SQLite.
- SQLite.
- HTML simple.

Frase que define este proyecto:

Camus LDA es un sistema web para gestionar una empresa de alcantarillado. El frontend en React muestra los módulos del negocio; el backend en Node.js expone una API REST; PostgreSQL guarda los datos. Hoy los módulos de clientes, órdenes y parte del inventario ya están conectados a la base de datos; el resto funciona como prototipo visual con datos de prueba.